

ISG022

PROJETO INTEGRADOR

Segurança de Aplicações

Desenvolvimento incremental • Times ágeis • Sprints de 2 semanas • GitHub

Documento de referência do projeto semestral. O produto evolui junto com as aulas: cada novo conteúdo de Segurança da Informação deve gerar uma decisão, evidência, controle, teste ou melhoria incorporada ao projeto.

FATEC • 2º semestre de 2026

1. Visão do Projeto

O Projeto Integrador de ISG022 será desenvolvido durante o semestre por equipes de cinco estudantes. Cada equipe deverá conceber, implementar, testar, documentar e defender uma aplicação Web e/ou API na qual os conceitos de segurança sejam incorporados progressivamente ao ciclo de desenvolvimento.

O projeto não deve ser tratado como uma aplicação pronta à qual a equipe adicionará segurança no final. A proposta é trabalhar Security by Design de forma incremental: o domínio nasce simples e, a cada bloco da disciplina, novas ameaças, requisitos, decisões arquiteturais, controles e testes passam a fazer parte do produto.

Princípio central: conteúdo visto em aula → análise no projeto → item de backlog → implementação/evidência → revisão na Sprint.

2. Objetivos

- Aplicar os fundamentos de Segurança da Informação a um produto de software real.
- Desenvolver capacidade de identificar ativos, ameaças, vulnerabilidades, riscos e requisitos de segurança.
- Tomar decisões arquiteturais justificadas e rastreáveis.
- Implementar controles de autenticação, autorização, proteção de dados, validação, logging e tratamento seguro de erros.
- Aplicar práticas e testes relacionados às vulnerabilidades estudadas na disciplina, incluindo referências OWASP.
- Trabalhar em equipe com organização ágil, papéis definidos, backlog e entregas incrementais.
- Utilizar Git e GitHub como mecanismo de colaboração, versionamento, revisão e evidência da evolução do projeto.
- Comunicar e defender tecnicamente as decisões de segurança adotadas.

3. Organização das Equipes

Cada equipe será formada por 5 integrantes, com a seguinte estrutura:

Papel	Quantidade	Responsabilidade principal
Product Manager (PM)	1	Organiza o backlog, mantém visão do produto, acompanha prioridades, critérios de aceite e comunicação da Sprint.
Tech Lead (TL)	1	Coordena decisões técnicas, arquitetura, padrões de implementação, integração, qualidade técnica e apoio ao time.
Team Members	3	Implementam, testam, documentam, revisam código e participam das decisões e entregas da Sprint.

Os papéis representam responsabilidades de organização, não hierarquia acadêmica. Todos os integrantes devem compreender o produto, o código e as decisões de segurança. O professor poderá solicitar explicações a qualquer integrante durante reviews e apresentações.

A rotação de papéis poderá ser adotada durante o semestre para ampliar a experiência dos integrantes, conforme orientação do professor.

4. Método de Trabalho: Ágil com Sprints de 2 Semanas

O desenvolvimento será organizado em ciclos de duas semanas. Cada Sprint deve possuir objetivo claro, itens priorizados, responsáveis, evidências de execução e uma entrega demonstrável.

1. Planejar	2. Construir	3. Testar	4. Revisar	5. Apresentar / Registrar
-------------	--------------	-----------	------------	---------------------------

Cerimônias e checkpoints mínimos

- Sprint Planning: definir Sprint Goal e selecionar itens do Product Backlog.
- Acompanhamento: registrar andamento, impedimentos e decisões durante a Sprint.
- Code Review: alterações relevantes devem ser revisadas por outro integrante antes do merge.
- Sprint Review: demonstrar o incremento produzido e as evidências de segurança.
- Retrospectiva: registrar pelo menos um ponto que funcionou e uma melhoria para o próximo ciclo.

5. GitHub como Plataforma do Projeto

Cada grupo manterá um repositório GitHub como fonte oficial do projeto. O histórico do repositório será parte da evidência de participação e evolução incremental.

Estrutura mínima recomendada

- README.md — visão do produto, integrantes, papéis, instruções de execução e status.
- docs/ — arquitetura, requisitos de segurança, threat model, decisões e evidências.
- src/ ou estrutura equivalente — código da aplicação.
- tests/ — testes automatizados e/ou scripts de validação.
- .github/ — templates, workflows e configurações de colaboração, quando aplicável.

Fluxo de colaboração recomendado

- Product Backlog registrado em GitHub Issues e/ou GitHub Projects.
- Uma Issue deve representar uma tarefa, requisito, vulnerabilidade, melhoria ou dívida relevante.
- Desenvolvimento em branches de curta duração; evitar trabalho permanente diretamente na branch principal.
- Pull Request para integração de mudanças relevantes.
- Review por pelo menos outro integrante sempre que possível.
- Commits pequenos, descritivos e rastreáveis ao trabalho realizado.
- Tags/releases ou marcos equivalentes para identificar incrementos importantes.

Segredos, senhas, tokens, chaves privadas e credenciais reais NÃO devem ser versionados.

Arquivos de ambiente devem usar valores de exemplo e mecanismos adequados de configuração.

6. Evolução Incremental do Projeto

A sequência abaixo é o roadmap-base do projeto. As Sprints acompanham a progressão didática da disciplina; o professor poderá ajustar datas ou escopo sem alterar a lógica incremental.

Etapa	Foco	O que evolui	Evidências mínimas
Sprint 0	Fundação	Domínio, problema, usuários, formação do grupo, papéis, repositório, backlog inicial e visão do produto.	README inicial; equipe/papéis; repositório; backlog.
Sprint 1	Fundamentos e riscos	Inventário de ativos; CIA/autenticidade; ameaças; vulnerabilidades; riscos; requisitos iniciais de segurança.	Inventário de ativos; matriz inicial de riscos; requisitos de segurança; backlog atualizado.
Sprint 2	Arquitetura segura	Aplicar princípios de projeto seguro e modelar arquitetura, fronteiras de confiança e ameaças.	Diagrama de arquitetura; threat model; decisões arquiteturais; controles planejados.
Sprint 3	Identidade e acesso	Implementar autenticação, autorização e gestão de sessão adequadas ao domínio.	Fluxos de autenticação/autorização; testes; evidências; revisão do threat model.
Sprint 4	Mitigações OWASP	Evoluir entradas, consultas, saídas e fluxos expostos conforme vulnerabilidades estudadas.	Correções implementadas; testes de segurança; evidências antes/depois.
Sprint 5	Observabilidade e testes	Logging, auditoria, tratamento de erros, testes e análise de dependências/configurações.	Logs seguros; evidências de auditoria; suíte de testes; achados e correções.
Sprint 6	Hardening e defesa	Revisão final de segurança, riscos residuais, documentação, qualidade e preparação da defesa.	Release candidata; checklist; riscos residuais; documentação final; roteiro de demonstração.
Entrega Final	Produto e defesa	Aplicação funcional, repositório organizado, documentação consolidada e apresentação técnica.	Release final; documentação; apresentação/demonstração; defesa das decisões.

7. Como cada aula alimenta o projeto

O backlog não será estático. Após conteúdos relevantes, a equipe deverá perguntar: “o que aprendemos hoje muda algo no nosso produto?”. Quando a resposta for sim, deve surgir uma ação rastreável: requisito, Issue, decisão, teste, correção ou risco aceito.

Conteúdo da disciplina	Reflexo esperado no projeto
CIA, autenticidade, ativos e riscos	Identificar ativos e propriedades de segurança; registrar ameaças e riscos.
Princípios de projeto seguro	Revisar arquitetura e decisões de acesso, exposição e comportamento em falhas.
Threat Modeling	Modelar ameaças, fronteiras de confiança e caminhos de ataque plausíveis.
Autenticação e autorização	Implementar identidade, papéis/permissoes e controles server-side.
OWASP / injeção / XSS / validação	Criar testes e mitigações aplicáveis ao produto.
Logs, auditoria e erros	Garantir rastreabilidade sem vaziar dados sensíveis.
Testes e hardening	Validar controles, dependências, configurações e riscos residuais.

8. Artefatos Obrigatórios

- Repositório GitHub com histórico de evolução.
- README completo e atualizado.
- Product Backlog e histórico de Sprints.
- Inventário de ativos e requisitos de segurança.
- Registro/matriz de riscos e respectivas decisões de tratamento.
- Modelo de ameaças e arquitetura da aplicação.
- Código-fonte da aplicação e configurações necessárias para execução.
- Testes e evidências das principais medidas de segurança.
- Registro de decisões técnicas relevantes (ADRs ou formato equivalente recomendado).
- Documentação final com riscos residuais conhecidos.
- Apresentação e demonstração final.

9. Definition of Ready (DoR)

Um item deve estar suficientemente preparado antes de entrar na Sprint. Como referência:

- Objetivo e contexto compreensíveis.
- Critério de aceite definido.
- Dependências conhecidas.
- Impacto de segurança identificado quando aplicável.
- Tamanho compatível com a Sprint ou decomposto em itens menores.

10. Definition of Done (DoD)

Um item somente deve ser considerado concluído quando, conforme aplicável:

- Implementação está integrada e executável.
- Critérios de aceite foram atendidos.
- Código foi revisado.
- Testes relevantes foram executados.
- Não foram adicionados segredos ao repositório.
- Documentação necessária foi atualizada.
- Impactos/riscos de segurança foram avaliados.
- Issue/PR contém evidência suficiente para demonstrar a entrega.

11. Requisitos mínimos do produto final

O domínio é escolhido pelo grupo, mas o produto final deve possuir complexidade suficiente para permitir análise e aplicação dos conceitos da disciplina. Como baseline, espera-se uma aplicação Web/API com usuários e dados persistentes, regras de acesso e operações que permitam demonstrar controles de segurança.

- Pelo menos dois perfis ou contextos de autorização com permissões distintas.
- Persistência de dados relevante ao domínio.
- Autenticação e autorização implementadas de forma coerente.
- Validação de entradas e tratamento seguro das operações.
- Proteção adequada de credenciais/segregados.
- Logging/auditoria de eventos relevantes sem exposição indevida de informações sensíveis.
- Tratamento de erros que não exponha detalhes desnecessários.
- Testes de segurança associados aos riscos e vulnerabilidades trabalhados em aula.
- Documentação de riscos mitigados, aceitos ou ainda residuais.

O professor poderá adequar requisitos técnicos conforme a natureza do domínio escolhido e o conteúdo efetivamente trabalhado ao longo do semestre.

12. Evidência de participação individual

Embora o produto seja coletivo, a participação é individualmente observável. O GitHub e as cerimônias devem permitir identificar contribuição técnica, organizacional e documental. Quantidade de commits isoladamente não representa qualidade nem participação.

- Issues assumidas e concluídas.
- Commits e Pull Requests com conteúdo significativo.
- Reviews realizados em contribuições de colegas.
- Participação em documentação, testes e análise de segurança.
- Capacidade de explicar decisões e trechos do produto durante reviews e defesa.

13. Avaliação do Projeto

Conforme o planejamento da disciplina, o Projeto Integrador corresponde a 30% da média final. A avaliação deve considerar o produto e o processo, não apenas a quantidade de funcionalidades.

Dimensão	O que será observado
Evolução incremental e processo ágil	Backlog, Sprints, rastreabilidade, organização e constância de entrega.
Engenharia e funcionamento do produto	Coerência da solução, qualidade técnica e funcionamento demonstrável.
Segurança aplicada	Qualidade da análise de riscos e adequação dos controles implementados.
Testes e evidências	Capacidade de provar que controles e requisitos foram verificados.
GitHub e colaboração	Uso consistente de versionamento, Issues/PRs/reviews e participação da equipe.

Documentação e defesa

Clareza, justificativa de decisões, riscos residuais e domínio técnico na apresentação.

A distribuição detalhada de pontos por dimensão poderá ser publicada em rubrica específica antes da entrega final.

14. Regras de Segurança e Ética

- Testes ofensivos devem ocorrer somente no ambiente do próprio projeto ou em ambientes explicitamente autorizados pelo professor.
- É proibido atacar, sondar ou tentar explorar sistemas, contas, redes ou serviços de terceiros.
- Não utilizar dados pessoais reais quando dados fictícios forem suficientes.
- Não versionar credenciais ou segredos reais.
- Vulnerabilidades encontradas em ambientes institucionais ou de terceiros devem ser comunicadas ao professor e não exploradas.
- Ferramentas de IA podem apoiar o trabalho conforme orientação da disciplina, mas o grupo continua responsável por compreender, revisar e defender tudo o que entregar.

15. Entrega Final e Defesa

Na apresentação final, o grupo deverá demonstrar o produto e contar a história de sua evolução de segurança.

- Problema e domínio da aplicação.
- Arquitetura e principais ativos.
- Ameaças/riscos mais relevantes identificados.
- Decisões de segurança adotadas e justificativas.
- Demonstração de controles e testes.
- Exemplos de vulnerabilidades evitadas/corrigidas.
- Riscos residuais e limitações conhecidas.
- Evolução do backlog e principais aprendizados das Sprints.

A defesa não é uma leitura de slides. Qualquer integrante pode ser questionado sobre arquitetura, código, risco, teste, Issue, Pull Request ou decisão do grupo.

16. Checklist da Sprint 0

Para iniciar o projeto, cada grupo deve sair da Sprint 0 com:

- 5 integrantes definidos.
- 1 PM, 1 Tech Lead e 3 Team Members definidos para o ciclo inicial.
- Nome provisório do projeto.
- Domínio/problema descrito em poucas linhas.
- Usuários principais identificados.
- Repositório GitHub criado.
- README inicial.

- Backlog inicial com épicos/funcionalidades e primeiras tarefas.
- Primeiros ativos relevantes identificados.
- Acordo básico de trabalho do grupo: comunicação, branches, PRs e reviews.

17. Exemplo de item de backlog de segurança

US-SEC-003 — Como usuário autenticado, quero que somente eu e perfis autorizados consigamos consultar meus dados acadêmicos, para evitar exposição indevida de informações.

Critérios de aceite sugeridos:

- Usuário não autenticado recebe resposta de acesso negado.
- Usuário autenticado não consegue consultar o recurso de outro usuário sem permissão.
- A verificação de autorização ocorre no servidor/API.
- Tentativas negadas relevantes são registradas sem expor dados sensíveis.
- Existe teste demonstrando ao menos um cenário permitido e um cenário negado.

18. Resultado esperado

Ao final do semestre, o repositório deverá contar a história do projeto: como a equipe saiu de uma ideia de aplicação, identificou o que precisava proteger, modelou ameaças, tomou decisões, implementou controles, encontrou problemas, corrigiu vulnerabilidades e chegou a uma versão final cujo nível de segurança pode ser explicado e demonstrado.

Segurança não será um capítulo final do projeto. Será parte do processo de engenharia desde a primeira Sprint.